

Roteiro do Trabalho, Dia 2: ligar o frontend na API

Este documento assume que o Dia 1 terminou com as 15 rotas da API funcionando e testadas, com commits já enviados pro seu fork. Hoje o trabalho é ligar o site nessas rotas.

Conceitos que você vai usar hoje

- **Frontend:** a parte que roda dentro do navegador do usuário. HTML é a estrutura da página, CSS é a aparência, JavaScript é o comportamento.
- **Consumir uma API:** fazer o frontend pedir dados pra API (o backend que você terminou no Dia 1) em vez de escrever os dados direto na página.
- **fetch:** a função do JavaScript que faz uma requisição HTTP pra uma URL, o mesmo que o navegador faz ao abrir um site, só que de dentro do seu código.
- **Assíncrono, async/await:** uma requisição pela rede leva um tempo pra responder. `async/await` é o jeito do JavaScript esperar essa resposta chegar sem travar a página inteira enquanto espera.
- **resposta.json():** a resposta da API chega como texto no formato JSON. Esse método transforma esse texto num objeto JavaScript que o código consegue usar.
- **DOM:** a representação da página HTML que o JavaScript consegue ler e alterar. `document.getElementById` busca um elemento da página pra colocar dados dentro dele.
- **Console do navegador:** painel de erros e mensagens. Abra com F12, ou botão direito na página > Inspeccionar > Console. É o primeiro lugar pra olhar quando uma página não carrega os dados.

1. Subir os dois servidores juntos

Terminal 1 (API, já devia estar de pé do dia anterior):

```
cd material-alunos/backend
source .venv/bin/activate
python3 app.py
```

Terminal 2 (site estático):

```
cd material-alunos/frontend
python3 -m http.server 5500
```

Abra `http://127.0.0.1:5500` no navegador. Cada página vai mostrar um erro de "TODO: implemente" na tela, porque `js/api.js` ainda não faz nenhuma chamada real. Isso é esperado, é o ponto de partida do dia.

Problema comum no Mac: se o erro no navegador for "Load failed" em vez do TODO, o Safari pode estar resolvendo `localhost` pra IPv6 e caindo no serviço de AirPlay do macOS, que também usa a porta 5000. Confira em `js/config.js` se `API_BASE_URL` está com `http://127.0.0.1:5000/api` (não `localhost`). Isso evita o problema em qualquer navegador, não só no Safari.

2. Implementar `js/api.js`

Toda a comunicação com o backend passa por esse arquivo só. Cada função tem um comentário dizendo o método HTTP e a rota. Como você já testou todas as 15 rotas no dia anterior, aqui é só chamar `_apiRequest` com o método e o caminho certo, usando `await`. Exemplo, a primeira função já mostra o padrão:

```
async function apiListarCategorias() {  
  return _apiRequest("GET", "/categorias");  
}
```

As que recebem parâmetro na URL usam template string:

```
async function apiObterProduto(id) {  
  return _apiRequest("GET", `/produtos/${id}`);  
}
```

As que mandam corpo (POST) passam o terceiro argumento:

```
async function apiCriarCliente(dados) {  
  return _apiRequest("POST", "/clientes", dados);  
}
```

E a que monta query string (`apiListarProdutos`) usa `new URLSearchParams`, removendo antes as chaves vazias/undefined pra não mandar `categoria_id=undefined` na URL.

Cada função de `api.js` chama a rota que você já implementou no Dia 1, e cada rota corresponde ao mesmo exercício do `bdc2` que você resolveu antes. Use essa tabela pra saber exatamente qual exercício revisar se a chamada não funcionar:

bdc2	função em <code>api.js</code>
ecom04	<code>apiListarCategorias</code>
ecom05 / ecom06	<code>apiListarProdutos</code>
ecom07	<code>apiListarDestques</code>
ecom08	<code>apiObterProduto</code>
ecom09	<code>apiListarAvaliacoes</code>
ecom10	<code>apiCriarAvaliacao</code>

bdc2	função em api.js
ecom11	apiCriarCliente
ecom12	apiLogin
ecom13	apiListarPedidosCliente
ecom14	apiValidarCupom
ecom15	apiCriarPedido
ecom16	apiObterPedido
ecom17	apiRelatorioResumo
ecom18	apiRelatorioMaisVendidos
ecom19	apiRelatorioFaturamentoPorCategoria

Vá função por função, testando a página correspondente no navegador antes de seguir pra próxima. Ordem sugerida:

Bloco 1, catálogo (ecom04 a ecom10): `apiListarCategorias`, `apiListarProdutos`, `apiListarDestaques`, `apiObterProduto`, `apiListarAvaliacaoes`, `apiCriarAvaliacao`. Teste abrindo `index.html` e `produto.html`, o catálogo tem que aparecer com os produtos reais do seu banco.

```
git add material-alunos/frontend/js/api.js
git commit -m "frontend: catalogo consumindo a api"
git push
```

Bloco 2, conta (ecom11 a ecom13): `apiCriarCliente`, `apiLogin`, `apiListarPedidosCliente`. Teste cadastro e login em `cadastro.html` e `login.html`.

```
git add material-alunos/frontend/js/api.js
git commit -m "frontend: cadastro e login consumindo a api"
git push
```

Bloco 3, compra (ecom14 a ecom16): `apiValidarCupom`, `apiCriarPedido`, `apiObterPedido`. Teste o fluxo completo: adicionar produto no carrinho, ir pro checkout, aplicar um cupom válido, fechar o pedido, e confirmar que o pedido aparece em `conta.html`.

```
git add material-alunos/frontend/js/api.js
git commit -m "frontend: carrinho e checkout consumindo a api"
git push
```

Bloco 4, painel (ecom17 a ecom19): `apiRelatorioResumo`, `apiRelatorioMaisVendidos`, `apiRelatorioFaturamentoPorCategoria`. Teste `painel.html`.

```
git add material-alunos/frontend/js/api.js
git commit -m "frontend: painel da loja consumindo a api"
git push
```

3. Personalizar o tema

Em `js/config.js`: nome da loja e slogan. Em `css/style.css`: cores. Não é o foco do trabalho, mas leva 10 minutos e deixa o resultado com a cara do seu grupo.

```
git add material-alunos/frontend/js/config.js material-alunos/frontend/css/style.css
git commit -m "frontend: personalizacao visual do tema"
git push
```

4. Teste de ponta a ponta, o teste final

Sem isso o trabalho não está pronto, mesmo que cada peça tenha funcionado separada:

1. Abrir o catálogo, filtrar por categoria, buscar por nome.
2. Abrir um produto, ler as avaliações, deixar uma avaliação nova.
3. Cadastrar um cliente novo.
4. Fazer login com esse cliente.
5. Adicionar 2 ou 3 produtos ao carrinho.
6. Ir pro checkout, aplicar um cupom válido, fechar o pedido.
7. Conferir em `conta.html` que o pedido aparece no histórico.
8. Abrir `painel.html` e confirmar que o relatório de mais vendidos mudou com o pedido que você acabou de fazer.

Se algum passo quebrar, volta pro dia anterior: o problema quase sempre está na query, não no JavaScript.

Checklist de fim do Dia 2 (entrega)

- Os 15 endpoints estão sendo chamados pelo frontend, nenhuma tela mostra erro de "TODO" ou "Load failed".
- O fluxo de compra completo (cadastro, login, carrinho, checkout, pedido) funciona sem recarregar a página manualmente.
- O painel da loja reflete os pedidos reais do banco.
- Todos os commits dos dois dias estão no seu fork, com `git push` confirmado (`git log --oneline` mostra o histórico, `git status` está limpo).
- Link do seu fork enviado pro professor.